/*
* Describes a transaction attribute on an individual method or on a class.
* If no custom rollback rules are configured, the transaction will roll back
* on RuntimeException and Error but not on checked exception
*/
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Inherited
@Documented
public @interface Transactional {

…

Propagation propagation() default Propagation.REQUIRED;

Isolation isolation() default Isolation.DEFAULT;

int timeout() default TransactionDefinition.TIMEOUT_DEFAULT;

Class<? extends Throwable>[] rollbackFor() default {};

Class<? extends Throwable>[] noRollbackFor() default {};

…

}

@Service
public class UserServiceImpl implements UserService{
@Autowired
private UserDao dao;

@Transactional(propagation = Propagation.REQUIRED, //传播
isolation = Isolation.DEFAULT, //使⽤底层DB的默认隔离级别
timeout = 30,
rollbackFor = Exception.class)
@Override
public void modifyUsersWithTransaction() { //有事务控制，test1和test2都⼊库失败
User u = new User();
u.setName("test3");
u.setAge(20);
u.setPhoneNum("123456789");
u.setAddress("test3");
dao.saveUser(u);

int i = 4/0; //故意制造exception

u = new User();
u.setName("test4");
u.setAge(20);
u.setPhoneNum("123456789");
u.setAddress("test4");
dao.saveUser(u);
}
}

@SpringBootTest
public class TransactionTest {
@Autowired
UserService userServiceImpl;

@Test
public void testTransaction1() {
userServiceImpl.modifyUsers();
}

@Test
public void testTransaction2() {
userServiceImpl.modifyUsersWithTransaction();
}
}

Object CglibAopProxy$DynamicAdvisedInterceptor.intercept(Object proxy, Method method, Object[] args, MethodProxy methodProxy) throws Throwable {
Object oldProxy = null; boolean setProxyContext = false; Object target = null;
TargetSource targetSource = this.advised.getTargetSource();
target = targetSource.getTarget();
Class<?> targetClass = (target != null ? target.getClass() : null);
List<Object> chain = this.advised.getInterceptorsAndDynamicInterceptionAdvice(method, targetClass); //获取拦截器链
Object retVal;
if (chain.isEmpty() && CglibAopProxy.CglibMethodInvocation.isMethodProxyCompatible(method)) {
Object[] argsToUse = AopProxyUtils.adaptArgumentsIfNecessary(method, args);
retVal = invokeMethod(target, method, argsToUse, methodProxy); //CASE1:不带@Transactional的⽅法，直接执⾏⽬标⽅法
}
//CASE2:带@Transactional的⽅法，沿着拦截器链调⽤
else retVal = new CglibMethodInvocation(proxy, target, method, args, targetClass, chain, methodProxy).proceed();
retVal = processReturnType(proxy, target, method, retVal);
return retVal;
}

Object ReflectiveMethodInvocation.proceed() {
//2.执⾏业务逻辑

if (this.currentInterceptorIndex == this.interceptorsAndDynamicMethodMatchers.size() - 1) return invokeJoinpoint();
//1.执⾏拦截器

Object interceptorOrInterceptionAdvice = this.interceptorsAndDynamicMethodMatchers.get(++this.currentInterceptorIndex);
return ((MethodInterceptor) interceptorOrInterceptionAdvice).invoke(this);
}

Object TranslationInterceptor.invoke(MethodInvocation invocation) {
Class<?> targetClass = invocation.getThis() != null ? AopUtils.getTargetClass(invocation.getThis()) : null;
return this.invokeWithinTransaction(invocation.getMethod(), targetClass, new CoroutinesInvocationCallback() {
public Object proceedWithInvocation() throws Throwable { return invocation.proceed(); }
public Object getTarget() { return invocation.getThis(); }
public Object[] getArguments() { return invocation.getArguments(); }
});
}

Object ReflectiveMethodInvocation.invokeJoinpoint() throws Throwable {
AopUtils.invokeJoinpointUsingReflection(this.target, this.method, this.arguments);
}
AopUtils.invokeJoinpointUsingReflection(Object target, Method method, Object[] args) throws Throwable {
try {
ReflectionUtils.makeAccessible(method);
return method.invoke(target, args);
}
}

public class UserServiceImpl$$EnhancerBySpringCGLIB$$96369935 extends UserServiceImpl implements SpringProxy, Advised, Factory {
private MethodInterceptor CGLIB$CALLBACK_0;

public final void modifyUsers() {
MethodInterceptor var10000 = this.CGLIB$CALLBACK_0;
if (var10000 != null) {
var10000.intercept(this, CGLIB$modifyUsers$3$Method, CGLIB$emptyArgs, CGLIB$modifyUsers$3$Proxy);
}
else super.modifyUsers();
}

public final void modifyUsersWithTransaction() {
MethodInterceptor var10000 = this.CGLIB$CALLBACK_0;
if (var10000 != null) {
var10000.intercept(this, CGLIB$modifyUsersWithTransaction3Method, CGLIB$emptyArgs, CGLIB$modifyUsersWithTransaction3Proxy);
}
else super.modifyUsersWithTransaction();
}
}

Object TransactionAspectSupport.invokeWithinTransaction(Method method, Class<?> targetClass, final InvocationCallback invocation) {
//获取@Transactional的属性值（例如，传播级别、隔离级别等）
TransactionAttribute txAttr = tas != null ? tas.getTransactionAttribute(method, targetClass) : null;
//开启事务
TransactionAspectSupport.TransactionInfo txInfo = this.createTransactionIfNecessary(ptm, txAttr, joinpointIdentification);
Object retVal;
try { retVal = invocation.proceedWithInvocation(); } //执⾏业务逻辑
//根据业务逻辑执⾏情况决定commit or rollback
catch (Throwable ex) {
completeTransactionAfterThrowing(txInfo, ex); //CASE1：业务逻辑抛异常，回滚
throw ex;
}
finally { cleanupTransactionInfo(txInfo); }
commitTransactionAfterReturning(txInfo); //CASE2：业务逻辑正常结束，提交
return retVal;
}

void completeTransactionAfterThrowing(TransactionAspectSupport.TransactionInfo txInfo, Throwable ex)
{
if (txInfo != null && txInfo.getTransactionStatus() != null) {
if (txInfo.transactionAttribute != null && txInfo.transactionAttribute.rollbackOn(ex)) {
txInfo.getTransactionManager().rollback(txInfo.getTransactionStatus());
} else txInfo.getTransactionManager().commit(txInfo.getTransactionStatus());
}
}

… - JdbcTransactionManager.doRollback(DefaultTransactionStatus status) {
DataSourceTransactionObject txObject = DataSourceTransactionObject)status.getTransaction();
Connection con = txObject.getConnectionHolder().getConnection();
con.rollback();
}

void completeTransactionAfterReturning(TransactionAspectSupport.TransactionInfo txInfo) {
if (txInfo != null && txInfo.getTransactionStatus() != null) {
txInfo.getTransactionManager().commit(txInfo.getTransactionStatus());
}
}

… - JdbcTransactionManager.doCommit(DefaultTransactionStatus status) {
DataSourceTransactionObject txObject = DataSourceTransactionObject)status.getTransaction();
Connection con = txObject.getConnectionHolder().getConnection();
con.commit();
}
//commit之后还会执⾏cleanupAfterCompletion()，清理当前线程本地变量⾥保存的资源

/* 1. 获取数据库连接，关闭autocommit
* 2. 将当前连接和当前线程绑定（缓存到当前线程的本地变量中），后续CRUD操作获取连接时，都⽤这个 */
TransactionInfo createTransactionIfNecessary(PlatformTransactionManager tm,TransactionAttribute txAttr) {
TransactionStatus status = tm.getTransaction(txAttr);
return prepareTransactionInfo(tm, txAttr, joinpointIdentification, status);
}

Object invokeMethod(Object target, Method method, Object[] args, MethodProxy methodProxy) throws Throwable {
try { return methodProxy.invoke(target, args); } //1. 先尝试⽤CGLIB的MethodProxy执⾏
catch (CodeGenerationException ex) { //2.fast-class⽣成失败时，直接反射调⽤
return AopUtils.invokeJoinpointUsingReflection(target, method, args);
}
}

@Service
public class UserServiceImpl implements UserService{
@Autowired
private UserDao dao;

@Override
public void modifyUsers() { //没有事务控制，test1⼊库成功，test2⼊库失败
User u = new User();
u.setName("test1");
u.setAge(20);
u.setPhoneNum("123456789");
u.setAddress("test1");
dao.saveUser(u);

int i = 4/0; //故意制造exception

u = new User();
u.setName("test2");
u.setAge(20);
u.setPhoneNum("123456789");
u.setAddress("test2");
dao.saveUser(u);
}
}

… - Object org.mybatis.spring.SqlSessionTemplate$SqlSessionInterceptor.invoke(Object proxy, Method method, Object[] args) throws Throwable {
SqlSession sqlSession = SqlSessionUtils.getSqlSession(SqlSessionTemplate.this.sqlSessionFactory, SqlSessionTemplate.this.executorType, SqlSessionTemplate.this.exceptionTranslator);
Object unwrapped;
try {
/* mybatis执⾏CRUD操作：
* > 如果开启了事务，使⽤创建事务时缓存到当前线程本地变量中的连接（autoCommit被关闭）
* > 否则，从数据源中重新获取连接（autoCommit是打开的，执⾏完直接commit到数据库）
Object result = method.invoke(sqlSession, args);
if (!SqlSessionUtils.isSqlSessionTransactional(sqlSession, SqlSessionTemplate.this.sqlSessionFactory)) {
sqlSession.commit(true); //如果没开启事务，且数据库连接没有autoCommit，mybatis会执⾏connection.commit()
}
unwrapped = result;
} catch (Throwable var11) { … }
finally { if (sqlSession != null) SqlSessionUtils.closeSqlSession(sqlSession, SqlSessionTemplate.this.sqlSessionFactory); }
return unwrapped;
}

SqlSession getSqlSession(SqlSessionFactory sessionFactory, ExecutorType executorType, PersistenceExceptionTranslator exceptionTranslator) {
SqlSessionHolder holder = (SqlSessionHolder)TransactionSynchronizationManager.getResource(sessionFactory);
SqlSession session = sessionHolder(executorType, holder);
if (session != null) return session; //当前事务已创建session，直接返回
else {
session = sessionFactory.openSession(executorType);
//如果开启了事务（tm.getTransaction()时设置事务标志），将session保存⾄当前线程本地变量
if (TransactionSynchronizationManager.isSynchronizationActive()) registerSessionHolder(sessionFactory, executorType, exceptionTranslator, session);
return session;
}
}

… - SqlSession DefaultSqlSessionFactory.openSessionFromDataSource(ExecutorType execType, TransactionIsolationLevel level, boolean autoCommit) { //autoCommit=false
Transaction tx = null; DefaultSqlSession var8;
try {
Environment environment = this.configuration.getEnvironment();
TransactionFactory transactionFactory = this.getTransactionFactoryFromEnvironment(environment);
tx = transactionFactory.newTransaction(environment.getDataSource(), level, autoCommit); //new JdbcTransaction(ds, level, autoCommit);
Executor executor = this.configuration.newExecutor(tx, execType);
var8 = new DefaultSqlSession(this.configuration, executor, autoCommit);
} catch (Exception var12) { this.closeTransaction(tx); } finally { ErrorContext.instance().reset(); }
return var8;
}

void registerSessionHolder(SqlSessionFactory sessionFactory, ExecutorType executorType, PersistenceExceptionTranslator exceptionTranslator, SqlSession session){
SqlSessionHolder holder = new SqlSessionHolder(session, executorType, exceptionTranslator);
TransactionSynchronizationManager.bindResource(sessionFactory, holder); //session和当前线程绑定
TransactionSynchronizationManager.registerSynchronization(new SqlSessionUtils.SqlSessionSynchronization(holder, sessionFactory));
holder.setSynchronizedWithTransaction(true);
holder.requested();
}

boolean isSqlSessionTransactional(SqlSession session, SqlSessionFactory sessionFactory) {
SqlSessionHolder holder = (SqlSessionHolder)TransactionSynchronizationManager.getResource(sessionFactory);
return holder != null && holder.getSqlSession() == session;
}

1. SpringBoot启动
执⾏SpringBootBasicApplication.main() -> 创建SpringApplication实例，执⾏run()
> 创建ApplicationContext（AnnotationConfigServletWebServerApplicationContext）
> prepareContext（注册springBootBasicApplication的beanDefinition）
> refreshContext
> invokeBeanFactoryPostProcessors ⾃动装配时注册事务切⾯bean definition
> onRefresh（创建WebServer）
> finishBeanFactoryInitialization 创建动态代理
> finishRefresh（启动WebServer）
> callRunners

2. SpringBoot响应请求
> Tomcat⾸次监听到http请求时，初始化spring mvc九⼤组件
> 根据路径匹配到springboot应⽤，开始DispatcherServlet.doDispatch()
1.getHandler
2.getHandlerAdaper
3.invoke handlerMethod

> connector从response缓冲区获取json内容，⽣成http响应报⽂

创建bean userServiceImpl：
1. 加载com.javadevmap.service.UserServiceImpl.java
2. resolveBeforeInstantiation
3. doCreateBean
> createBeanInstance
> applyMergedBeanDefinitionPostProcessors
> populateBean 注⼊依赖
> initializeBean（bean完成初始化之后，开始执⾏AOP代理）
> registerDisposableBeanIfNecessary
4. proxy代替userServiceImpl注册到beanFactory.singletonObjects

AnnotationAwareAspectJAutoProxyCreator.postProcessAfterInitialization()
> 找出所有适⽤于当前bean的advisors
> create proxy（保存了所有筛选好的advisors）

> findCandidateAdvisors

> 筛选适⽤于当前bean的advisors：当前类和⽅法符合pointcut的执⾏条件
> 排序（增强代码的织⼊顺序）

BeanFactoryTransactionAttributeSourceAdvisor筛选逻辑：遍历targetClass UserServiceImpl的所有methods，发现带@Transactional的method，返回true

⾃动装配TransactionAutoConfiguration，解析@EnableTransactionManagement
> 引⼊TransactionManagementConfigurationSelector
> 引⼊ProxyTransactionManagementConfiguration
> 将BeanFactoryTransactionAttributeSourceAdvisor beanDefinition注⼊bean factory

遍历当前类的所有advisors，找出pointcut是当前method的，将所有advices组成执⾏链

UserController响应请求，调⽤userServiceImpl proxy.saveUser()

> 筛选拦截器：从proxy的advisors中找出pointCut是当前⽅法的，将所有advices组成拦截器链
> 如果有要执⾏的拦截器：沿着拦截器调⽤

> 如果没有要执⾏的拦截器：userServiceImpl其他不加@Transactional的⽅法，直接执⾏serServiceImpl.saveUser()

执⾏TranslationInterceptor：
> 获取@Transactional的属性值（传播级别、隔离级别等），开启事务
> 关闭数据库连接的autoCommit

> 执⾏业务逻辑
> 如果抛异常，connection.rollback()
> 如果正常结束，connection.commit() > cleanupAfterCompletion()

从dataSource中获取⼀个数据库连接，关闭autoCommit，保存到当前线程的本地变量中（后续mybatis执⾏CRUD操作也会⽤这个）

> 清理当前线程本地变量中和当前事务相关的资源
> 重置connection的autoCommit
> 释放connection

mybatis执⾏CRUD操作：
> 如果开启了事务，使⽤当前线程本地变量中的connection：CRUD执⾏完不会commit，交给spring控制
> 否则，从数据源中重新获取connection：autoCommit⼀般都是打开的，CRUD执⾏完直接commit到db（如果autoCommit是关闭的，mybatis会执⾏connection.commit()）

> 启动时context.refresh()
> invokeBeanFactoryPostProcessors(): ConfigurationClassPostProcessor⾃动装配TransactionAutoConfiguration时，注册BeanFactoryTransactionAttributeSourceAdvisor bean definition
> finishBeanFactoryInitialization(): 创建userServiceImpl单例后，AnnotationAwareAspectJAutoProxyCreator.postProcessAfterInitialization()，创建CGLib代理proxy，在业务逻辑前后织⼊事务控制逻辑

> 响应请求DispatcherServlet.doDispatch()
> UserController响应请求，调⽤userServiceImpl，被proxy拦截，执⾏增强的事务逻辑
> 开启事务
> 执⾏业务逻辑
> commit or rollback

enum Isolation { //隔离级别
DEFAULT(-1), //使⽤DB的默认隔离级别（MYSql默认使⽤Repeated Read，且InnoDB存储引擎解决了幻读）
READ_UNCOMMITTED(1), //可读取其他事务未提交的数据（脏读、幻读、不可重复读）
READ_COMMITTED(2), //只可读取其他事务提交的数据（幻读、不可重复读）
REPEATABLE_READ(4), //可重复⼀致地读取其他事务提交的数据（幻读）
SERIALIZABLE(8); //事务之间完全串⾏读取，每次读取都要获取表级共享锁
}

enum Propagation { //传播级别
REQUIRED, //Support a current transaction, create a new one if none exists.
SUPPORTS, //Support a current transaction, execute non-transactionally if none exists.
MANDATORY, //Support a current transaction, throw an exception if none exists.
REQUIRES_NEW, //Create a new transaction, and suspend the current transaction if one exists.
NOT_SUPPORTED, //Execute non-transactionally, suspend the current transaction if one exists.
NEVER, //Execute non-transactionally, throw an exception if a transaction exists.
NESTED //Execute within a nested transaction if a current transaction exists
}

该值最终被设置到statement对象中，控制sql执⾏时间，超过指定值抛异常，被spring事务切⾯捕获后rollback

mybatis-spring-boot-starter引⼊mybatis-spring依赖（spring集成mybatis⽤）
mybatis执⾏CRUD时，通过SpringManagedTransaction使⽤Spring的DataSourceUtils获取数据库连接

> beanFactory中Advisor实现类：BeanFactoryTransactionAttributeSourceAdvisor
> @Aspect声明的Advisors